

Peer-Discovery Methods in Pantheon Piracy Intelligence

How we measure per-title piracy demand — the three independent ways we find the people sharing a file, why we use all three, and why PEX is the slowest.

DHT

distributed lookup

Trackers

central directory · workhorse

PEX

peer-to-peer gossip · supplement

The core idea

Our demand signal for a title is simple: **the number of distinct IP addresses sharing that title over BitTorrent**, counted per country per day. An IP = a device/person downloading or seeding. More distinct IPs = more real-world demand.

The hard part is *finding* those IPs. BitTorrent has no central list of "everyone sharing file X." Instead there are three independent discovery mechanisms — and no single one sees everyone. So we run all three, then **merge and de-duplicate** the IPs. An IP found by two methods is counted once; an IP only one method found still counts. The union is a far more complete demand number than any single source.

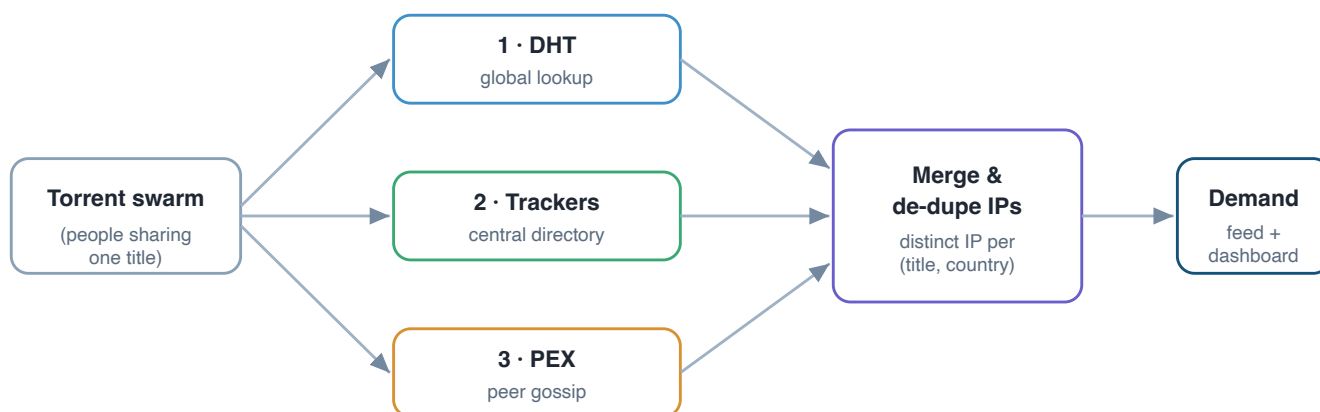


Figure 1 — One swarm, three independent discovery methods, merged into one de-duplicated demand number.

The DHT is a **global, server-less phone book** spread across millions of BitTorrent clients. Every torrent is identified by a 20-byte *infohash*. The DHT lets any client ask the network "who is sharing infohash X?" without any central server — the answer is stored across the nodes whose IDs are numerically *closest* to that infohash.

How a lookup works

You start from a few bootstrap nodes and send `get_peers(infohash)`. Each node replies either with peers it knows for that infohash, or with a list of *other nodes closer* to the infohash. You repeatedly query the closer nodes — an iterative "walk" that converges on the nodes holding the peer list. We sample those peers to count IPs.

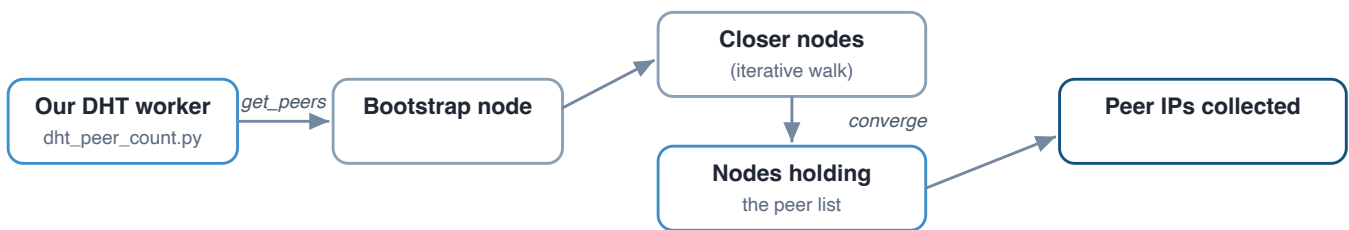


Figure 2 — DHT lookup: an iterative walk toward the nodes that hold the peer list for an infohash.

How we use it. 8 always-on worker processes (`dht_peer_count.py`) cycle through every tracked infohash, doing `get_peers` walks and recording the peer IPs (geolocated to a country). Tuned this session: `DHT_ROUND_MS=400` + an "early-stop" hop-limit so workers stop walking once a hash stops yielding new peers.

Strengths: fully decentralized, can't be shut down, finds peers no tracker lists. **Weaknesses:** only a *sample* of the swarm (you reach the nodes near the infohash, not every peer) — so DHT alone undercounts. In our data DHT sees ~32% of all distinct IPs.

Tracker Harvest (announce, BEP-3 / BEP-15)

A **tracker** is a central server that keeps the live list of everyone in a torrent's swarm. When a client joins, it sends an **announce** to the tracker ("I have infohash X, here's my IP/port") and the tracker replies with a chunk of current peers. It's the original, fastest way to discover peers — one request returns many IPs.

How harvesting works

Our harvester behaves like a client joining the swarm: for each infohash it sends **announce** requests to a large list of public trackers (UDP and HTTP) and collects every peer the trackers return. Because trackers keep the authoritative live list, this yields the most IPs per request — it's our workhorse.

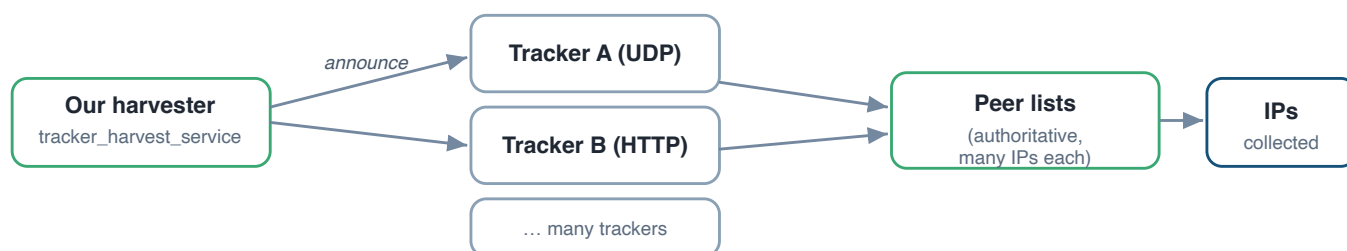


Figure 3 — Tracker harvest: one announce per tracker returns a large chunk of the live swarm.

How we use it. A continuous service (`tracker_harvest_service.py`) announces every infohash to a broad tracker list and writes peers to a separate database. A high-velocity lane re-harvests brand-new releases more often to catch day-0 churn.

Strengths: fastest, highest yield — the authoritative live list. **Weaknesses:** depends on trackers being up and listing the torrent; private/dead trackers and DHT-only swarms are invisible to it. In our data the harvester sees ~88% of all distinct IPs — the workhorse.

PEX is **word-of-mouth between peers**. Once you're connected to a peer, it periodically tells you about *other* peers it's connected to — no server, no DHT involved. It's a gossip layer that rides on top of connections you already have.

Why it needs a connection first (and why that makes it slow)

To receive PEX gossip from a peer you must complete a full sequence per peer: open a **TCP connection** → do the **BitTorrent handshake** → do the **extension (LTP) handshake** advertising `ut_pex` → then **wait** for the peer to send periodic `ut_pex` messages (roughly once a minute, capped count). Compare that to DHT (one UDP query) or a tracker (one announce) returning many peers immediately.



Figure 4 — PEX: gossip layered on top of full per-peer connections. It cannot find a swarm cold; it can only relay peers a connected peer already knows.

How we use it. A service (`pex_harvest.py`) takes infohashes from the harvester, connects to known peers, completes the handshakes advertising `ut_pex` , and records the gossiped peer IPs as a 4th source.

Why it is slow / how to grow it. PEX is *secondary* — it is bootstrapped from peers we already found, needs a heavy connect+handshake per peer, and gossip is rate-limited. More worker processes help only up to the box's connection/bandwidth limit; the bigger levers are **covering more infohashes** (it runs on ~51 today) and **holding connections longer** for more gossip rounds. It will always be the smallest source — but ~42% of the IPs it finds are *unique* (no other method saw them), which is why it earns its place.

Side-by-side comparison

	DHT (BEP-5)	Trackers (announce)	PEX (BEP-11)
What it is	Server-less global lookup	Central live directory	Peer-to-peer gossip
How peers arrive	Iterative get_peers walk	One announce → many peers	Periodic gossip from a connected peer
Needs a connection first?	No (UDP query)	No (one request)	Yes — TCP + 2 handshakes per peer
Speed	Medium	Fast	Slow (sequential, rate-limited)
Can find a swarm cold?	Yes	Yes	No — must be seeded by the others
Our component	dht_peer_count.py x8	tracker_harvest_service.py	pex_harvest.py
Share of distinct IPs*	~32%	~88%	~0.2% (growing)
Role	Coverage booster	Workhorse	Unique-lift supplement

*Measured on our 2026-06-07 feed. Shares overlap (the same IP is often seen by more than one method) so they sum to >100% — that overlap is exactly why we de-duplicate. On the hashes PEX covers, ~42% of its IPs were found by nobody else.

Why use all three?

No single method is complete. Trackers miss DHT-only swarms; DHT only samples the swarm; PEX reaches pockets of peers the others never connected to. Running all three and de-duplicating yields a materially larger, more trustworthy demand count than any one source — DHT alone adds ~12.5% of IPs that trackers miss, and PEX adds a smaller but genuinely-unique slice.

Further reading & videos

Official specifications

- DHT Protocol — [BEP-5](#)
- Tracker / announce — [BEP-3](#), UDP trackers [BEP-15](#)
- Peer Exchange (ut_pex) — [BEP-11](#); Extension Protocol [BEP-10](#)

Explainer videos (YouTube)

- Torrents Explained: How BitTorrent Works — youtube.com/watch?v=urzQeD7ftbl
- BitTorrent & DHT Protocols — how do they work? — youtube.com/watch?v=xg0RHkuTwIQ
- A Deep Dive Into The BitTorrent Protocol — youtube.com/watch?v=dleYEIHuTQo
- How a client discovers peers using DHT (simple terms) — youtube.com/watch?v=nfeyY1ymXDc
- Introduction to BitTorrent and the problem it solves — youtube.com/watch?v=v7cR0ZolaUA